Machine Learning Practice Code Handbook

100 Essential Snippets · Industry Standard · Beginner to Advanced

---

EDA · Feature Engineering · Missing Values · Outliers

Feature Construction · Dimensionality Reduction

Tech Stack: Pandas · NumPy · Matplotlib · Seaborn · Scikit-learn

Crafted for self-paced skill-building from zero to production-grade ML

Table of Contents

CHAPTER 1

Exploratory Data Analysis (EDA)

Univariate & Multivariate techniques to understand your data before modelling

#01 Load & First Look at Data

Always start here. Get shape, column names, dtypes, and a peek at actual values to understand what you're working with.

```python
import pandas as pd
import numpy as np

df = pd.read_csv('data.csv')

print('Shape:', df.shape) # rows x cols
print('Columns:', df.columns.tolist())
print('Dtypes:\n', df.dtypes)
print('\nFirst 5 rows:')
print(df.head())
print('\nLast 5 rows:')
print(df.tail())
```

#02 Descriptive Statistics — Numerical

describe() gives you count, mean, std, min, quartiles, max in one shot. Use include='all' to get stats for categoricals too.

```python
# Full descriptive statistics
print(df.describe()) # numeric columns
print(df.describe(include='all')) # all columns

# Individual stats
print('Mean:\n', df.mean(numeric_only=True))
print('Median:\n', df.median(numeric_only=True))
print('Std:\n', df.std(numeric_only=True))
print('Skewness:\n', df.skew(numeric_only=True)) # important for ML
print('Kurtosis:\n', df.kurt(numeric_only=True))
```

#03 Missing Value Analysis

Before any modelling, quantify missingness per column. The percentage matters — below 5% is usually safe to impute simply.

```python
# Count and percentage of missing values
missing = df.isnull().sum()
missing_pct = (missing / len(df)) * 100

missing_df = pd.DataFrame({
'Missing Count': missing,
'Missing %': missing_pct.round(2)
}).sort_values('Missing %', ascending=False)

print(missing_df[missing_df['Missing Count'] > 0])
```

#04 Univariate — Distribution of a Numeric Column

Histogram reveals skewness. KDE (density) curve shows the smooth shape. Use both side-by-side routinely.

```python
```

```python
import matplotlib.pyplot as plt
import seaborn as sns

col = 'age' # change to your column

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Histogram
axes[0].hist(df[col].dropna(), bins=30, color='steelblue', edgecolor='white')
axes[0].set_title(f'Histogram: {col}')
axes[0].set_xlabel(col); axes[0].set_ylabel('Frequency')

# KDE plot
sns.kdeplot(df[col].dropna(), ax=axes[1], fill=True, color='steelblue')
axes[1].set_title(f'KDE: {col}')

plt.tight_layout()
plt.show()
```

#05 Univariate — Categorical Value Counts

Always inspect cardinality (number of unique values) before encoding. High-cardinality needs special treatment.

```python
col = 'city' # change to your column

# Frequency table
print(df[col].value_counts())
print('\nUnique values:', df[col].nunique())

# Bar chart
plt.figure(figsize=(10, 4))
df[col].value_counts().plot(kind='bar', color='steelblue', edgecolor='white')
plt.title(f'Value Counts: {col}')
plt.ylabel('Count')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

#06 Univariate — Boxplot for Spread & Outliers

Boxplots are perfect for quickly spotting outliers and understanding IQR. Points beyond whiskers are potential outliers.

```python
col = 'salary'

plt.figure(figsize=(8, 3))
sns.boxplot(x=df[col].dropna(), color='lightblue')
plt.title(f'Boxplot: {col}')
plt.tight_layout()
plt.show()

# Print the 5-number summary
q1 = df[col].quantile(0.25)
q3 = df[col].quantile(0.75)
iqr = q3 - q1
print(f'Q1={q1:.2f} Median={df[col].median():.2f} Q3={q3:.2f} IQR={iqr:.2f}')
```

#07 Univariate — Violin Plot (Distribution + Spread)

Violin plot combines KDE and boxplot. Wider sections mean more data density — very informative.

```python
```

```
plt.figure(figsize=(8, 4))
sns.violinplot(y=df['age'].dropna(), color='mediumpurple', inner='box')
plt.title('Violin Plot: Age Distribution')
plt.tight_layout()
plt.show()
```

#08 Univariate — QQ Plot (Normality Check)

Points falling on the diagonal line mean the column is normally distributed — important for models that assume normality.

```python
from scipy import stats

col = 'income'

plt.figure(figsize=(6, 5))
stats.probplot(df[col].dropna(), dist='norm', plot=plt)
plt.title(f'Q-Q Plot: {col}')
plt.tight_layout()
plt.show()

# Formal normality test
stat, p = stats.shapiro(df[col].dropna().sample(min(5000, len(df))))
print(f'Shapiro p-value: {p:.4f} (normal if p > 0.05)')
```

#09 Univariate — All Numeric Histograms at Once

A fast overview of ALL numeric columns simultaneously — use this on every new dataset.

```python
df.hist(figsize=(16, 10), bins=30, color='steelblue',
edgecolor='white', layout=None)
plt.suptitle('All Numeric Feature Distributions', y=1.01, fontsize=14)
plt.tight_layout()
plt.show()
```

#10 Multivariate — Correlation Heatmap

The heatmap reveals linear relationships between features. Features with high correlation to the target are useful; highly correlated features with each other cause multicollinearity.

```python
plt.figure(figsize=(12, 8))
corr_matrix = df.select_dtypes(include='number').corr()

sns.heatmap(
corr_matrix,
annot=True, # show correlation values
fmt='.2f',
cmap='coolwarm', # red=positive, blue=negative
center=0,
linewidths=0.5,
square=True
)
plt.title('Feature Correlation Heatmap')
plt.tight_layout()
plt.show()
```

#11 Multivariate — Scatter Plot (Two Features)

Scatter plots reveal non-linear patterns, clusters, and outliers between two continuous variables.

```python
```

```python
plt.figure(figsize=(7, 5))
plt.scatter(df['age'], df['income'], alpha=0.4, color='steelblue', edgecolors='white', s=30)
plt.xlabel('Age'); plt.ylabel('Income')
plt.title('Age vs Income')
plt.tight_layout()
plt.show()

# With regression line
sns.regplot(data=df, x='age', y='income', scatter_kws={'alpha': 0.4}, line_kws={'color': 'red'})
plt.title('Age vs Income with Regression Line')
plt.show()
```

#12 Multivariate — Pairplot (All Feature Pairs)

Pairplot is one of the most powerful EDA tools — gives scatter for every pair, histograms on the diagonal, and can color by target.

```python
# Select a subset of columns for readability
cols = ['age', 'income', 'score', 'target']

sns.pairplot(
df[cols].dropna(),
hue='target', # color by target class
diag_kind='kde', # KDE on diagonal
plot_kws={'alpha': 0.5}
)
plt.suptitle('Pairplot', y=1.01)
plt.show()
```

#13 Multivariate — Grouped Statistics

Group by a categorical variable to understand how numeric features differ across groups — essential for feature importance intuition.

```python
# Mean of numeric cols grouped by target
print(df.groupby('target').mean(numeric_only=True))

# Box plots grouped by category
plt.figure(figsize=(10, 5))
sns.boxplot(data=df, x='target', y='income', palette='Set2')
plt.title('Income Distribution by Target Class')
plt.tight_layout()
plt.show()
```

#14 Multivariate — Heatmap of Missing Values

msno (missingno) creates a visual map of where nulls live — great for spotting patterns in missingness (MCAR, MAR, MNAR).

```python
# pip install missingno
import missingno as msno

# Matrix view: white = missing
msno.matrix(df, figsize=(12, 5), fontsize=10)
plt.title('Missing Value Matrix')
plt.show()

# Correlation of missingness
msno.heatmap(df, figsize=(10, 6))
plt.title('Missing Value Correlation')
plt.show()
```

■ *Run: pip install missingno before using this snippet.*

#15 Multivariate — Crosstab (Two Categorical Features)

crosstab counts co-occurrences of two categorical variables. Normalize to get proportions.

```python
ct = pd.crosstab(df['gender'], df['churn'],
margins=True, normalize='index') # row proportions
print(ct.round(3))

# Visualise as heatmap
plt.figure(figsize=(6, 4))
sns.heatmap(pd.crosstab(df['gender'], df['churn']),
annot=True, fmt='d', cmap='Blues')
plt.title('Gender vs Churn Crosstab')
plt.show()
```

#16 Multivariate — Feature vs Target (Numeric Target)

When target is continuous, scatter each feature against it to see relationships before building a model.

```python
num_cols = df.select_dtypes(include='number').columns.tolist()
num_cols = [c for c in num_cols if c != 'target']

n = len(num_cols)
fig, axes = plt.subplots((n + 2) // 3, 3, figsize=(15, 4 * ((n + 2) // 3)))
axes = axes.flatten()

for i, col in enumerate(num_cols):
axes[i].scatter(df[col], df['target'], alpha=0.3, s=10)
axes[i].set_xlabel(col)
axes[i].set_ylabel('target')
axes[i].set_title(f'{col} vs target')

# Hide unused axes
for j in range(i + 1, len(axes)):
axes[j].set_visible(False)

plt.tight_layout()
plt.show()
```

#17 Multivariate — Count Plot (Categorical vs Target)

Count plots with hue reveal class imbalances within each category — very useful for classification tasks.

```python
plt.figure(figsize=(10, 4))
sns.countplot(data=df, x='education', hue='target', palette='Set1')
plt.title('Education Level by Target Class')
plt.xticks(rotation=30, ha='right')
plt.tight_layout()
plt.show()
```

#18 Multivariate — Scatter Matrix (Color-coded)

Using pandas scatter_matrix for a quick multi-feature look with alpha control for dense datasets.

```python
```

```
from pandas.plotting import scatter_matrix

cols = ['feature1', 'feature2', 'feature3', 'feature4']

scatter_matrix(
df[cols],
figsize=(10, 8),
alpha=0.3,
diagonal='hist',
color='steelblue'
)
plt.suptitle('Scatter Matrix', y=1.01)
plt.show()
```

#19 Multivariate — Facet Grid Analysis

FacetGrid creates subplots for each level of a categorical variable — perfect for comparing distributions.

```python
g = sns.FacetGrid(df, col='region', col_wrap=3, height=3)
g.map(sns.histplot, 'income', bins=20, color='steelblue')
g.set_titles(col_template='{col_name}')
g.set_axis_labels('Income', 'Count')
plt.suptitle('Income by Region', y=1.02)
plt.show()
```

#20 EDA — Automated Summary Report

Generate a comprehensive profiling report for the entire dataset with one function call.

```python
# pip install ydata-profiling
from ydata_profiling import ProfileReport

profile = ProfileReport(df, title='Dataset EDA Report',
explorative=True,
minimal=False) # set True for large datasets

# Save to HTML
profile.to_file('eda_report.html')
print('Report saved to eda_report.html')
```

■ *Run: pip install ydata-profiling. Great for a quick full-dataset summary.*

CHAPTER 2

Feature Engineering

Scaling, Encoding, Pipelines, Mixed Variables, Date-Time — transforming raw data into ML-ready features

#21 Standardization (Z-score Scaling)

StandardScaler makes mean=0, std=1. Use this for algorithms sensitive to feature scale: SVM, KNN, PCA, Logistic Regression, Neural Nets.

```python
from sklearn.preprocessing import StandardScaler
import pandas as pd

scaler = StandardScaler()

# Fit on TRAIN only — then transform both train and test
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test) # NOT fit_transform!

# Recover a DataFrame with column names
X_train_df = pd.DataFrame(X_train_scaled, columns=X_train.columns)

print('Mean after scaling (should be ~0):', X_train_df.mean().round(4).values)
print('Std after scaling (should be ~1):', X_train_df.std().round(4).values)
```

■ *NEVER call fit_transform on test data — that causes data leakage.*

#22 Normalization (Min-Max Scaling)

MinMaxScaler squeezes all values into [0, 1]. Best for neural networks and when you need bounded output.

```python
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler(feature_range=(0, 1)) # can also use (0, 1) or (-1, 1)

X_train_norm = scaler.fit_transform(X_train)
X_test_norm = scaler.transform(X_test)

print('Min after scaling:', X_train_norm.min(axis=0).round(4))
print('Max after scaling:', X_train_norm.max(axis=0).round(4))

# Inverse transform to recover original values
X_original = scaler.inverse_transform(X_train_norm)
```

#23 Robust Scaler (Outlier-Resistant Scaling)

RobustScaler uses median and IQR instead of mean and std. Perfect when your data has outliers you don't want to remove.

```python
from sklearn.preprocessing import RobustScaler

scaler = RobustScaler(quantile_range=(25.0, 75.0)) # uses IQR

X_train_r = scaler.fit_transform(X_train)
X_test_r = scaler.transform(X_test)

# Outliers are now scaled but not eliminated
print('Median after transform (approx 0):',
pd.DataFrame(X_train_r).median().round(3).values)
```

#24 Ordinal Encoding

Use OrdinalEncoder when categories have a natural order: Low < Medium < High. Preserves the ordering information as integers.

```python
from sklearn.preprocessing import OrdinalEncoder
import numpy as np

# Define the order explicitly — very important!
categories = [['Low', 'Medium', 'High']]

enc = OrdinalEncoder(categories=categories,
handle_unknown='use_encoded_value',
unknown_value=-1)

df['education_encoded'] = enc.fit_transform(df[['education']])

print(df[['education', 'education_encoded']].drop_duplicates())
```

#25 One-Hot Encoding (sklearn)

OneHotEncoder converts categories to binary columns. Use drop='first' to avoid the dummy variable trap in linear models.

```python
from sklearn.preprocessing import OneHotEncoder
import pandas as pd

enc = OneHotEncoder(drop='first', # avoids multicollinearity
sparse_output=False, # returns dense array
handle_unknown='ignore')

encoded = enc.fit_transform(df[['city']])

# Reconstruct into a DataFrame
col_names = enc.get_feature_names_out(['city'])
df_enc = pd.DataFrame(encoded, columns=col_names, index=df.index)

df = pd.concat([df.drop('city', axis=1), df_enc], axis=1)
print(df.head())
```

#26 One-Hot Encoding with pandas get_dummies

pd.get_dummies is quick and convenient for notebooks, but sklearn's encoder is better for production pipelines.

```python
# Quick pandas OHE
df_dummies = pd.get_dummies(
df,
columns=['city', 'gender'], # columns to encode
drop_first=True, # remove first category
dtype=int # use int instead of bool
)
print(df_dummies.head())
```

#27 ColumnTransformer — Different Transforms per Column Type

ColumnTransformer is the industry-standard way to apply different preprocessing steps to different subsets of columns simultaneously.

```python
```

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

num_cols = ['age', 'income', 'score']
cat_cols = ['city', 'gender']

preprocessor = ColumnTransformer(transformers=[
('num', StandardScaler(), num_cols),
('cat', OneHotEncoder(drop='first',
handle_unknown='ignore'), cat_cols),
], remainder='passthrough') # keep other cols unchanged

X_train_proc = preprocessor.fit_transform(X_train)
X_test_proc = preprocessor.transform(X_test)
print('Processed shape:', X_train_proc.shape)
```

#28 FunctionTransformer — Apply Any Custom Function

FunctionTransformer wraps any Python/NumPy function into an sklearn-compatible transformer. Ideal for log transforms.

python

```
from sklearn.preprocessing import FunctionTransformer
import numpy as np

# Log transform to reduce right skew
log_transformer = FunctionTransformer(
func=np.log1p, # log(1+x) — handles zero values
inverse_func=np.expm1 # for inverse_transform support
)

X_log = log_transformer.fit_transform(df[['income', 'salary']])

# Square-root transform
sqrt_transformer = FunctionTransformer(np.sqrt)
X_sqrt = sqrt_transformer.fit_transform(df[['count_col']])
```

#29 Full ML Pipeline with Pipeline()

Pipeline chains preprocessing + model into a single object. This is the most important sklearn pattern — prevents data leakage and makes code production-ready.

python

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

# One unified object: scaler + model
pipe = Pipeline(steps=[
('scaler', StandardScaler()),
('model', LogisticRegression(max_iter=1000))
])

pipe.fit(X_train, y_train) # fits scaler AND model together
preds = pipe.predict(X_test) # scaler applied automatically
print('Accuracy:', pipe.score(X_test, y_test).round(4))
```

#30 Full Pipeline with ColumnTransformer + Model

The gold standard ML pipeline: preprocessing (ColumnTransformer) + model, all in one Pipeline object.

python

```python
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestClassifier

num_cols = ['age', 'income']
cat_cols = ['city', 'gender']

preprocessor = ColumnTransformer([
('num', StandardScaler(), num_cols),
('cat', OneHotEncoder(handle_unknown='ignore'), cat_cols),
])

full_pipe = Pipeline([
('prep', preprocessor),
('model', RandomForestClassifier(n_estimators=100, random_state=42))
])

full_pipe.fit(X_train, y_train)
print('Test Accuracy:', full_pipe.score(X_test, y_test).round(4))
```

#31 Target Encoding (Mean Encoding)

Target encoding replaces each category with the mean of the target for that category. Powerful but risky — use cross-validation to avoid leakage.

```python
# pip install category_encoders
import category_encoders as ce

encoder = ce.TargetEncoder(cols=['city'], smoothing=1.0)

# Must pass y for fitting
X_train_enc = encoder.fit_transform(X_train, y_train)
X_test_enc = encoder.transform(X_test)

print(X_train_enc['city'].head(10))
```

■ *Target encoding can leak target info. Always compute it inside a cross-validation fold.*

#32 Handling Mixed Variables — Splitting Numeric & Categorical

Mixed variables (e.g., '5 years', 'N/A', '100+') need to be parsed carefully before encoding or scaling.

```python
# Example: column has values like '30', '40+', 'Unknown', NaN
df['age_raw'] = ['30', '40+', 'Unknown', '25', None, '55']

import numpy as np

def clean_mixed_age(val):
if pd.isnull(val): return np.nan
val = str(val).strip()
if val in ('Unknown', 'N/A', ''): return np.nan
val = val.replace('+', '').replace('<', '') # strip symbols
try: return float(val)
except: return np.nan

df['age_clean'] = df['age_raw'].apply(clean_mixed_age)
print(df[['age_raw', 'age_clean']])
```

#33 Mixed Variables — Extracting Flag + Numeric Part

When a mixed column carries both a flag (like 'Unknown') and a numeric value, extract two separate features from it.

```python
# Original column: ['100', '200+', 'Refused', NaN, '50']
df['income_raw'] = ['100', '200', 'Refused', None, '50']

# Feature 1: was the value refused? (binary flag)
df['income_refused'] = df['income_raw'].apply(
lambda x: 1 if str(x).strip() == 'Refused' else 0
)

# Feature 2: numeric part
df['income_num'] = pd.to_numeric(
df['income_raw'].replace('Refused', np.nan), errors='coerce'
)

print(df[['income_raw', 'income_refused', 'income_num']])
```

#34 DateTime — Extracting Components

DateTime columns are useless as strings. Always extract year, month, day, dayofweek, hour, etc. as separate numeric features.

```python
df['date'] = pd.to_datetime(df['date']) # ensure datetime type

df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
df['day'] = df['date'].dt.day
df['dayofweek'] = df['date'].dt.dayofweek # 0=Monday
df['is_weekend'] = df['date'].dt.dayofweek.isin([5, 6]).astype(int)
df['hour'] = df['date'].dt.hour
df['quarter'] = df['date'].dt.quarter
df['week'] = df['date'].dt.isocalendar().week.astype(int)

print(df[['date','year','month','day','dayofweek','is_weekend']].head())
```

#35 DateTime — Days Since Reference Date (Feature)

Creating a numeric 'days since' feature is much more useful for models than raw dates.

```python
import pandas as pd

df['signup_date'] = pd.to_datetime(df['signup_date'])
df['last_login'] = pd.to_datetime(df['last_login'])
reference_date = pd.Timestamp('2024-01-01')

# Days since signup (useful for churn models)
df['days_since_signup'] = (reference_date - df['signup_date']).dt.days

# Days between two events
df['days_since_login'] = (reference_date - df['last_login']).dt.days

print(df[['signup_date', 'last_login',
'days_since_signup', 'days_since_login']].head())
```

#36 DateTime — Cyclic Encoding (Sin/Cos Transform)

Months and hours are cyclic: December (12) is close to January (1). Encode them with sin/cos to preserve this circularity.

```python
```

```
import numpy as np

# Month encoding (1-12 are cyclic)
df['month_sin'] = np.sin(2 * np.pi * df['month'] / 12)
df['month_cos'] = np.cos(2 * np.pi * df['month'] / 12)

# Hour encoding (0-23 are cyclic)
df['hour_sin'] = np.sin(2 * np.pi * df['hour'] / 24)
df['hour_cos'] = np.cos(2 * np.pi * df['hour'] / 24)

# Day of week encoding (0-6 are cyclic)
df['dow_sin'] = np.sin(2 * np.pi * df['dayofweek'] / 7)
df['dow_cos'] = np.cos(2 * np.pi * df['dayofweek'] / 7)

print(df[['month','month_sin','month_cos']].head())
```

#37 PowerTransformer (Yeo-Johnson) — Normalize Skewed Data

Yeo-Johnson makes skewed numeric distributions more Gaussian. Essential for linear models and PCA.

```python
from sklearn.preprocessing import PowerTransformer

pt = PowerTransformer(method='yeo-johnson', # handles negative values
standardize=True) # also standardizes

cols = ['income', 'spend', 'tenure']
df[cols + '_transformed'] = pt.fit_transform(df[cols])

# Quick before/after skewness comparison
print('Before skew:', df[cols].skew().round(3).values)
print('After skew: ',
pd.DataFrame(pt.fit_transform(df[cols]), columns=cols).skew().round(3).values)
```

#38 Binarizer — Threshold-based Binary Feature

Binarizer converts a numeric column into binary (0/1) based on a threshold. Useful for age groups, purchase amounts, etc.

```python
from sklearn.preprocessing import Binarizer

# Classify as 'high income' if income > 50000
binarizer = Binarizer(threshold=50000)

df['high_income'] = binarizer.fit_transform(df[['income']]).astype(int)

print(df[['income', 'high_income']].head(10))
```

#39 Label Encoding (for Tree-based Models)

LabelEncoder converts string categories to integers 0, 1, 2, … For tree models (RF, XGBoost) this is fine; for linear models, use OHE instead.

```python
```

```python
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df['city_encoded'] = le.fit_transform(df['city'].fillna('Unknown'))

# See the mapping
mapping = dict(zip(le.classes_, le.transform(le.classes_)))
print('Encoding map:', mapping)

# Decode back
df['city_decoded'] = le.inverse_transform(df['city_encoded'])
```

#40 Frequency Encoding

Frequency encoding replaces each category with how often it appears. Useful for high-cardinality columns.

```python
# Count frequency of each category
freq_map = df['city'].value_counts().to_dict()

df['city_freq'] = df['city'].map(freq_map)

# Normalize to proportions
df['city_freq_norm'] = df['city'].map(
df['city'].value_counts(normalize=True).to_dict()
)

print(df[['city', 'city_freq', 'city_freq_norm']].drop_duplicates())
```

#41 Pipeline with Cross-Validation (Best Practice)

Cross-validate the entire pipeline — this is the correct way. Never fit preprocessors outside CV.

```python
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
import numpy as np

pipe = Pipeline([
('scaler', StandardScaler()),
('svm', SVC(kernel='rbf', C=1.0))
])

# cv=5 means 5-fold cross-validation
scores = cross_val_score(pipe, X, y, cv=5, scoring='accuracy')
print(f'CV Accuracy: {scores.mean():.4f} +/- {scores.std():.4f}')
```

#42 Pipeline with GridSearchCV (Hyperparameter Tuning)

GridSearchCV with a Pipeline searches over both preprocessing parameters and model hyperparameters — the most powerful sklearn pattern.

```python
```

```
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

pipe = Pipeline([
('scaler', StandardScaler()),
('svc', SVC())
])

# Use double underscore to address pipeline step params
param_grid = {
'svc__C': [0.1, 1, 10, 100],
'svc__kernel': ['rbf', 'linear'],
'svc__gamma': ['scale', 'auto']
}

grid = GridSearchCV(pipe, param_grid, cv=5, n_jobs=-1, verbose=1)
grid.fit(X_train, y_train)
print('Best params:', grid.best_params_)
print('Best CV score:', grid.best_score_.round(4))
```

#43 Custom Transformer — sklearn-compatible Class

When sklearn doesn't have what you need, write your own transformer by inheriting from BaseEstimator and TransformerMixin.

```python
from sklearn.base import BaseEstimator, TransformerMixin
import numpy as np

class LogTransformer(BaseEstimator, TransformerMixin):
'''Apply log1p transform; inverse is expm1.'''
def fit(self, X, y=None):
return self # no fitting needed

def transform(self, X):
return np.log1p(X)

def inverse_transform(self, X):
return np.expm1(X)

# Use it in a Pipeline just like any sklearn transformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

pipe = Pipeline([('log', LogTransformer()), ('scale', StandardScaler())])
X_out = pipe.fit_transform(df[['income']])
```

#44 Mixed Variable — Housing Example (Numeric + Category + Date)

Real-world datasets combine all variable types. Here is a unified preprocessing pipeline for mixed data.

```python
```

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder, OrdinalEncoder
from sklearn.impute import SimpleImputer

num_pipe = Pipeline([
('impute', SimpleImputer(strategy='median')),
('scale', StandardScaler())
])

cat_pipe = Pipeline([
('impute', SimpleImputer(strategy='most_frequent')),
('encode', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])

ord_pipe = Pipeline([
('impute', SimpleImputer(strategy='most_frequent')),
('encode', OrdinalEncoder(categories=[['Poor','Fair','Good','Excellent']]))
])

full_prep = ColumnTransformer([
('num', num_pipe, num_cols),
('cat', cat_pipe, cat_cols),
('ord', ord_pipe, ord_cols)
])
```

#45 Get Feature Names After ColumnTransformer

After transforming, you often need column names back for interpretability and DataFrames.

```python
import pandas as pd

# After fitting the preprocessor
feature_names = preprocessor.get_feature_names_out()

X_train_df = pd.DataFrame(
preprocessor.transform(X_train),
columns=feature_names
)

print('Transformed features:', feature_names[:10])
print('Shape:', X_train_df.shape)
```

CHAPTER 3

Missing Value Imputation

Simple, Random Sample, KNN, and MICE (IterativeImputer) strategies for handling null values

#46 Understanding Missing Data Types

Knowing WHY data is missing guides HOW you impute. MCAR: missing at random. MAR: depends on other columns. MNAR: depends on the missing value itself.

```python
import pandas as pd
import numpy as np

# Summarise missingness
def missing_summary(df):
miss = df.isnull().sum()
pct = miss / len(df) * 100
dtype = df.dtypes
return pd.DataFrame({
'Missing': miss, 'Pct%': pct.round(2), 'Dtype': dtype
}).query('Missing > 0').sort_values('Pct%', ascending=False)

print(missing_summary(df))
```

#47 Simple Imputation — Mean / Median / Mode

SimpleImputer is the go-to for quick imputation. Use mean for symmetric, median for skewed numeric, most_frequent for categorical.

```python
from sklearn.impute import SimpleImputer
import pandas as pd

# Numeric: median (robust to outliers)
num_imputer = SimpleImputer(strategy='median')
df[num_cols] = num_imputer.fit_transform(df[num_cols])

# Categorical: most frequent (mode)
cat_imputer = SimpleImputer(strategy='most_frequent')
df[cat_cols] = cat_imputer.fit_transform(df[cat_cols])

# Constant: fill with a specific value
const_imputer = SimpleImputer(strategy='constant', fill_value='Missing')
df[['status']] = const_imputer.fit_transform(df[['status']])
```

#48 Simple Imputation — Mean Strategy

Use mean imputation for approximately symmetric distributions with low missingness (<5%).

```python
from sklearn.impute import SimpleImputer
import numpy as np

imputer = SimpleImputer(strategy='mean')

# Fit on train only — prevents data leakage
imputer.fit(X_train[num_cols])

X_train[num_cols] = imputer.transform(X_train[num_cols])
X_test[num_cols] = imputer.transform(X_test[num_cols])

print('Imputed means:', dict(zip(num_cols, imputer.statistics_.round(2))))
```

#49 Random Sample Imputation (Manual)

Random sample imputation preserves the original distribution of the feature — better than mean for skewed data.

```python
import pandas as pd
import numpy as np

def random_sample_impute(df, col):
'''Fill NaN with random samples drawn from observed values.'''
observed = df[col].dropna()
n_missing = df[col].isnull().sum()
random_samples = observed.sample(n_missing, replace=True).values
df.loc[df[col].isnull(), col] = random_samples
return df

# Apply to multiple columns
for col in ['age', 'income', 'score']:
df = random_sample_impute(df, col)

print('Missing after imputation:', df.isnull().sum().sum())
```

#50 Missing Indicator (Add Binary Flag Before Imputing)

ALWAYS add a binary flag before imputing — it captures the information that the value WAS missing, which can be predictive.

```python
from sklearn.impute import MissingIndicator, SimpleImputer
import pandas as pd
import numpy as np

# Step 1: create binary flags for missingness
indicator = MissingIndicator(features='missing-only')
missing_flags = indicator.fit_transform(X_train)

flag_cols = [f'{col}_was_missing'
for col in X_train.columns[indicator.features_]]
flags_df = pd.DataFrame(missing_flags.astype(int),
columns=flag_cols, index=X_train.index)

# Step 2: impute
imputer = SimpleImputer(strategy='median')
X_train_imp = pd.DataFrame(imputer.fit_transform(X_train),
columns=X_train.columns, index=X_train.index)

# Step 3: concatenate flags
X_train_final = pd.concat([X_train_imp, flags_df], axis=1)
print(X_train_final.shape)
```

#51 KNN Imputation

KNNImputer fills missing values using the K nearest neighbours in feature space. More accurate but slower than mean/median.

```python
```

```python
from sklearn.impute import KNNImputer
import pandas as pd

knn_imputer = KNNImputer(
n_neighbors=5, # use 5 nearest neighbours
weights='uniform', # or 'distance' to weight closer neighbours more
metric='nan_euclidean' # handles NaNs during distance calculation
)

X_train_knn = knn_imputer.fit_transform(X_train)
X_test_knn = knn_imputer.transform(X_test)

X_train_df = pd.DataFrame(X_train_knn, columns=X_train.columns)
print('KNN imputation complete. Missing:', X_train_df.isnull().sum().sum())
```

#52 KNN Imputation — Choosing K with Validation

Choosing the right K for KNNImputer can be done by evaluating model performance for each K on validation data.

```python
from sklearn.impute import KNNImputer
from sklearn.linear_model import Ridge
from sklearn.pipeline import Pipeline
from sklearn.model_selection import cross_val_score
import numpy as np

k_values = [3, 5, 7, 10, 15]
results = {}

for k in k_values:
pipe = Pipeline([
('impute', KNNImputer(n_neighbors=k)),
('model', Ridge())
])
scores = cross_val_score(pipe, X, y, cv=5, scoring='r2')
results[k] = scores.mean()
print(f'K={k:2d} -> R2: {scores.mean():.4f}')

best_k = max(results, key=results.get)
print(f'\nBest K: {best_k}')
```

#53 MICE Imputation — IterativeImputer

MICE (Multiple Imputation by Chained Equations) models each feature with missing values as a function of others. It's the most powerful imputation method.

```python
```

```python
from sklearn.experimental import enable_iterative_imputer # must import first
from sklearn.impute import IterativeImputer
from sklearn.linear_model import BayesianRidge
import pandas as pd

# BayesianRidge is the default estimator — works well for most cases
mice_imputer = IterativeImputer(
estimator=BayesianRidge(),
max_iter=10, # iterations of the chained equations
random_state=42,
imputation_order='ascending' # start from least missing
)

X_train_mice = mice_imputer.fit_transform(X_train)
X_test_mice = mice_imputer.transform(X_test)

X_train_df = pd.DataFrame(X_train_mice, columns=X_train.columns)
print('MICE done. Missing:', X_train_df.isnull().sum().sum())
```

#54 MICE with RandomForest Estimator

IterativeImputer can use any sklearn regressor/classifier as the imputation model — RandomForest handles non-linear relationships.

```python
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
from sklearn.ensemble import RandomForestRegressor

rf_imputer = IterativeImputer(
estimator=RandomForestRegressor(
n_estimators=50, random_state=42, n_jobs=-1
),
max_iter=5,
random_state=42
)

X_train_rf = rf_imputer.fit_transform(X_train)
X_test_rf = rf_imputer.transform(X_test)

print('RF-MICE imputation done.')
print('Shape:', X_train_rf.shape)
```

#55 Compare Imputation Strategies

Benchmark multiple strategies on downstream model performance to pick the best one for your dataset.

```python
```

```
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import SimpleImputer, KNNImputer, IterativeImputer
from sklearn.linear_model import Ridge
from sklearn.pipeline import Pipeline
from sklearn.model_selection import cross_val_score

strategies = {
'Mean': SimpleImputer(strategy='mean'),
'Median': SimpleImputer(strategy='median'),
'KNN-5': KNNImputer(n_neighbors=5),
'MICE': IterativeImputer(max_iter=10, random_state=42)
}

for name, imputer in strategies.items():
pipe = Pipeline([('imp', imputer), ('reg', Ridge())])
r2 = cross_val_score(pipe, X, y, cv=5, scoring='r2').mean()
print(f'{name:8s} -> R2: {r2:.4f}')
```

#56 Pandas fillna — Quick Imputation

For exploratory work, pandas fillna() is fast. Always prefer sklearn imputers inside ML pipelines.

python

```
# Fill with a fixed value
df['age'].fillna(df['age'].median(), inplace=True)

# Forward fill (carry previous value forward)
df['price'].fillna(method='ffill', inplace=True)

# Backward fill
df['price'].fillna(method='bfill', inplace=True)

# Interpolate (linear by default)
df['temperature'].interpolate(method='linear', inplace=True)

# Fill multiple columns differently
fill_map = {'income': df['income'].mean(),
'status': 'Unknown',
'score': 0}
df.fillna(fill_map, inplace=True)
```

#57 Group-based Imputation (Fill by Group Mean)

Instead of global mean, use the mean within each group. E.g., fill age from the same occupation group.

python

```
# Fill 'income' with the mean income for each 'education' group
df['income'] = df.groupby('education')['income'].transform(
lambda x: x.fillna(x.mean())
)

# For remaining NaN (groups with all-NaN), use global median
df['income'].fillna(df['income'].median(), inplace=True)

print('Remaining NaN:', df['income'].isnull().sum())
```

#58 Imputation with Pipeline Inside Cross-Validation

The correct way to impute in production — imputer is fitted inside each fold, never on the full dataset.

python

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score

pipe = Pipeline([
('imputer', SimpleImputer(strategy='median')),
('model', GradientBoostingClassifier(n_estimators=100))
])

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(pipe, X, y, cv=cv, scoring='roc_auc')
print(f'ROC-AUC: {scores.mean():.4f} +/- {scores.std():.4f}')
```

#59 Visualise Before and After Imputation

Always plot the distribution before and after imputation — a good imputation should preserve the original shape.

python

```
import matplotlib.pyplot as plt
import seaborn as sns

col = 'income'
original = df[col].dropna()

# Impute with median
median_val = original.median()
imputed = df[col].fillna(median_val)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
sns.histplot(original, ax=axes[0], bins=30, color='steelblue')
axes[0].set_title(f'Before Imputation (n={len(original)})')

sns.histplot(imputed, ax=axes[1], bins=30, color='green')
axes[1].set_title(f'After Median Imputation (n={len(imputed)})')

plt.tight_layout()
plt.show()
```

#60 Impute Time-Series Data (Interpolation)

For time-ordered data, linear or spline interpolation respects the temporal nature better than mean imputation.

python

```
import pandas as pd

# Sort by time first — critical for time-series imputation
df = df.sort_values('date').reset_index(drop=True)

# Linear interpolation
df['price_linear'] = df['price'].interpolate(method='linear')

# Spline (smoother curve)
df['price_spline'] = df['price'].interpolate(method='spline', order=3)

# Time-weighted interpolation
df_ts = df.set_index('date')
df_ts['price_time'] = df_ts['price'].interpolate(method='time')
```

#61 Multivariate Imputation for Categoricals with OHE

IterativeImputer works on numeric data only. For mixed data, encode categoricals first, then impute, then decode.

```python
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
from sklearn.preprocessing import OrdinalEncoder
import numpy as np

# Step 1: Ordinal encode categoricals (preserves NaN as NaN)
enc = OrdinalEncoder(handle_unknown='use_encoded_value', unknown_value=np.nan)
cat_encoded = enc.fit_transform(df[cat_cols])

# Step 2: Combine numeric + encoded categorical
import numpy as np
X_combined = np.hstack([df[num_cols].values, cat_encoded])

# Step 3: MICE impute
mice = IterativeImputer(max_iter=10, random_state=42)
X_imputed = mice.fit_transform(X_combined)

print('All done. Any NaN remaining:', np.isnan(X_imputed).any())
```

#62 Imputation Quality Metric (MAE on Artificially Masked Values)

Evaluate how good your imputation is by masking known values, imputing, and measuring error.

```python
import numpy as np
from sklearn.metrics import mean_absolute_error
from sklearn.impute import KNNImputer

col = 'income'
complete_col = df[col].dropna().copy()

# Randomly mask 20% of known values
rng = np.random.default_rng(42)
mask_idx = rng.choice(len(complete_col), size=int(0.2 * len(complete_col)), replace=False)

masked = complete_col.copy()
masked.iloc[mask_idx] = np.nan

# Impute with KNN
imputer = KNNImputer(n_neighbors=5)
imputed = imputer.fit_transform(masked.values.reshape(-1, 1)).flatten()

mae = mean_absolute_error(complete_col.values, imputed)
print(f'Imputation MAE: {mae:.4f}')
```

CHAPTER 4

Outlier Detection & Handling

Z-Score, IQR Boxplot method, and Percentile capping for identifying and handling extreme values

#63 What Are Outliers? — Visual Detection

First, always visualise potential outliers before deciding how to handle them. Context determines if they're errors or real extreme values.

```python
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

cols = df.select_dtypes(include='number').columns
n = len(cols)
fig, axes = plt.subplots((n + 2) // 3, 3, figsize=(15, 4 * ((n+2)//3)))
axes = axes.flatten()

for i, col in enumerate(cols):
sns.boxplot(y=df[col].dropna(), ax=axes[i], color='lightcoral')
axes[i].set_title(col)

for j in range(i + 1, len(axes)):
axes[j].set_visible(False)
plt.suptitle('Boxplots — Outlier Detection', y=1.01)
plt.tight_layout(); plt.show()
```

#64 Z-Score Method — Detect Outliers

Z-score measures how many standard deviations a point is from the mean. Points beyond ±3 are typically considered outliers.

```python
from scipy import stats
import numpy as np

col = 'income'

z_scores = np.abs(stats.zscore(df[col].dropna()))

# Common threshold: |z| > 3
threshold = 3
outlier_mask = z_scores > threshold

print(f'Total outliers in {col}: {outlier_mask.sum()}')
print(f'Percentage: {outlier_mask.mean()*100:.2f}%')
print('Outlier values:', df[col].dropna()[outlier_mask].values)
```

#65 Z-Score Method — Remove Outliers

After detection, remove rows with outlier values across all numeric columns simultaneously.

```python
```

```python
from scipy import stats
import numpy as np

num_cols = df.select_dtypes(include='number').columns

# Compute z-scores for all numeric columns at once
z_scores = np.abs(stats.zscore(df[num_cols].dropna()))

# Keep only rows where ALL columns have |z| <= 3
mask = (z_scores <= 3).all(axis=1)

df_clean = df[num_cols][mask].copy()
print(f'Original: {len(df)} rows After removal: {len(df_clean)} rows')
print(f'Removed: {len(df) - len(df_clean)} rows ({(1-mask.mean())*100:.2f}%)')
```

#66 Z-Score — Cap (Winsorize) Instead of Remove

Capping replaces outliers with the boundary value rather than dropping rows — better when data is limited.

```python
import numpy as np
from scipy import stats

col = 'income'
z = stats.zscore(df[col].dropna())

lower = df[col].mean() - 3 * df[col].std()
upper = df[col].mean() + 3 * df[col].std()

df[col + '_capped'] = df[col].clip(lower=lower, upper=upper)

print(f'Original range: [{df[col].min():.1f}, {df[col].max():.1f}]')
print(f'Capped range: [{df[col+"_capped"].min():.1f},
{df[col+"_capped"].max():.1f}]')
```

#67 IQR Boxplot Method — Detect Outliers

IQR method is more robust than Z-score for non-normal distributions. Points below Q1-1.5*IQR or above Q3+1.5*IQR are outliers.

```python
def detect_outliers_iqr(df, col):
Q1 = df[col].quantile(0.25)
Q3 = df[col].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
outliers = df[(df[col] < lower) | (df[col] > upper)]
return outliers, lower, upper

col = 'salary'
outliers, lo, hi = detect_outliers_iqr(df, col)
print(f'IQR Bounds: [{lo:.2f}, {hi:.2f}]')
print(f'Outliers found: {len(outliers)} rows')
print(outliers[col].describe())
```

#68 IQR Method — Remove Outliers

Remove rows that contain IQR outliers in ANY numeric column — most conservative cleaning approach.

```python
```

```
num_cols = df.select_dtypes(include='number').columns
mask = pd.Series([True] * len(df), index=df.index)

for col in num_cols:
Q1 = df[col].quantile(0.25)
Q3 = df[col].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
col_mask = (df[col] >= lower) & (df[col] <= upper)
mask = mask & col_mask

df_clean = df[mask].copy()
print(f'Rows kept: {len(df_clean)} / {len(df)} ({mask.mean()*100:.1f}%)')
```

#69 IQR Method — Cap Outliers (Winsorization)

IQR capping clips values to the whisker bounds. This is the most popular outlier handling strategy in industry.

python

```
def cap_outliers_iqr(series, factor=1.5):
Q1 = series.quantile(0.25)
Q3 = series.quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - factor * IQR
upper = Q3 + factor * IQR
return series.clip(lower=lower, upper=upper)

# Apply to all numeric columns
num_cols = df.select_dtypes(include='number').columns
df_capped = df.copy()
df_capped[num_cols] = df_capped[num_cols].apply(cap_outliers_iqr)

# Verify
print('Before:', df['income'].describe()[['min','max']].round(1).values)
print('After: ', df_capped['income'].describe()[['min','max']].round(1).values)
```

#70 Percentile Method — Cap at Any Percentile

Percentile capping (Winsorizing) clips values below the p-th percentile and above the (100-p)-th percentile. Very flexible.

python

```
def percentile_cap(series, lower_pct=1, upper_pct=99):
'''Cap values at the given lower and upper percentiles.'''
lower = series.quantile(lower_pct / 100)
upper = series.quantile(upper_pct / 100)
return series.clip(lower=lower, upper=upper)

# Apply 1st-99th percentile capping
for col in df.select_dtypes(include='number').columns:
df[col] = percentile_cap(df[col], lower_pct=1, upper_pct=99)

print('Capping complete.')
print(df.describe().loc[['min', 'max']].round(2))
```

#71 Percentile Method — sklearn Winsorizer

Feature-engine's Winsorizer is the clean sklearn-compatible version of percentile capping for pipelines.

python

```
# pip install feature-engine
from feature_engine.outliers import Winsorizer

# Cap at 5th and 95th percentiles
wins = Winsorizer(
capping_method='iqr', # or 'gaussian' or 'percentiles'
tail='both',
fold=1.5, # 1.5 * IQR
variables=['age', 'income', 'score']
)

wins.fit(X_train)
X_train_w = wins.transform(X_train)
X_test_w = wins.transform(X_test)

print('Right limits:', wins.right_tail_caps_)
print('Left limits: ', wins.left_tail_caps_)
```

■ *Run: pip install feature-engine for production-grade outlier transformers.*

#72 Isolation Forest — Anomaly-based Outlier Detection

IsolationForest detects outliers by isolating anomalies in random subspaces — works well for high-dimensional data.

python

```
from sklearn.ensemble import IsolationForest
import pandas as pd

iso = IsolationForest(
contamination=0.05, # assume 5% outliers
n_estimators=100,
random_state=42
)

preds = iso.fit_predict(df[num_cols].dropna())
# -1 = outlier, +1 = inlier

df['is_outlier_iso'] = (preds == -1).astype(int)
print('Outliers detected:', df['is_outlier_iso'].sum())

# Remove outliers
df_clean = df[df['is_outlier_iso'] == 0].drop('is_outlier_iso', axis=1)
```

#73 Before vs After Outlier Handling Comparison

Always compare distributions before and after handling. Good outlier treatment should not distort the overall shape.

python

```
import matplotlib.pyplot as plt
import seaborn as sns

col = 'income'

# IQR cap
Q1 = df[col].quantile(0.25); Q3 = df[col].quantile(0.75)
IQR = Q3 - Q1
df_clean = df[col].clip(Q1 - 1.5*IQR, Q3 + 1.5*IQR)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

sns.boxplot(y=df[col], ax=axes[0], color='lightcoral')
axes[0].set_title('Before: ' + col)

sns.boxplot(y=df_clean, ax=axes[1], color='lightgreen')
axes[1].set_title('After IQR Cap: ' + col)

plt.tight_layout(); plt.show()
```

#74 Floor and Ceiling Transform (Custom Bounds)

Sometimes domain knowledge gives you exact valid ranges. Use those bounds directly to clip.

```python
# Domain-specific bounds (e.g., age 0-120, score 0-100)
bounds = {
'age': (0, 120),
'score': (0, 100),
'price': (0.01, None), # only a lower bound
}

for col, (lo, hi) in bounds.items():
if col in df.columns:
df[col] = df[col].clip(lower=lo, upper=hi)
print(f'{col}: clipped to [{lo}, {hi}]')
```

#75 Log Transform to Reduce Outlier Effect

Log transform compresses large values, which effectively reduces the impact of outliers without removing them.

```python
import numpy as np
import matplotlib.pyplot as plt

col = 'income'

# log1p handles zero values gracefully
df['income_log'] = np.log1p(df[col])

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
df[col].hist(bins=50, ax=axes[0], color='lightcoral', edgecolor='white')
axes[0].set_title('Original Income')

df['income_log'].hist(bins=50, ax=axes[1], color='lightgreen', edgecolor='white')
axes[1].set_title('Log1p Transformed Income')

plt.tight_layout(); plt.show()

print('Skew before:', df[col].skew().round(3))
print('Skew after: ', df['income_log'].skew().round(3))
```

CHAPTER 5

Feature Construction

Build new, informative features from existing ones — often the single biggest ML performance booster

#76 Interaction Features — Multiply Two Features

The product of two features can capture synergy that neither alone expresses. E.g., (hours_studied × sleep_hours) for exam score.

python

```
# Manual interaction terms
df['age_x_income'] = df['age'] * df['income']
df['score_x_tenure'] = df['score'] * df['tenure']

# Using sklearn PolynomialFeatures
from sklearn.preprocessing import PolynomialFeatures

poly = PolynomialFeatures(degree=2, interaction_only=True,
include_bias=False)
interaction_array = poly.fit_transform(df[['age', 'income', 'score']])

import pandas as pd
interaction_df = pd.DataFrame(
interaction_array,
columns=poly.get_feature_names_out(['age', 'income', 'score'])
)
print(interaction_df.head())
```

#77 Polynomial Features (degree=2)

PolynomialFeatures generates x, x^2 terms. Useful for capturing non-linear patterns in linear models.

python

```
from sklearn.preprocessing import PolynomialFeatures
import pandas as pd

poly = PolynomialFeatures(
degree=2,
interaction_only=False, # includes x^2, y^2 etc.
include_bias=False # no constant term
)

X_poly = poly.fit_transform(df[['age', 'income']])
X_poly_df = pd.DataFrame(
X_poly,
columns=poly.get_feature_names_out(['age', 'income'])
)

print('New features:', poly.get_feature_names_out(['age', 'income']))
print('Shape:', X_poly_df.shape)
```

#78 Ratio Features

Ratios often carry more signal than raw values. E.g., income-to-age ratio, profit margin, or click-through rate.

python

```python
# Finance/business domain ratios
df['income_per_age'] = df['income'] / (df['age'] + 1e-6) # avoid /0
df['score_per_month'] = df['score'] / (df['months'] + 1)

# E-commerce domain
df['conversion_rate'] = df['purchases'] / (df['visits'] + 1)
df['revenue_per_visit'] = df['revenue'] / (df['visits'] + 1)

# Fraud detection
df['txn_amount_ratio'] = df['amount'] / (df['avg_amount'] + 1e-6)

print(df[['income_per_age', 'conversion_rate']].describe().round(3))
```

#79 Binning / Discretization

Converting continuous features into bins (age groups, income brackets) can help tree models and adds domain interpretability.

```python
import pandas as pd

# Equal-width bins
df['age_group'] = pd.cut(
df['age'],
bins=[0, 18, 30, 45, 60, 100],
labels=['Teen', 'Young', 'Mid', 'Senior', 'Elder'],
right=True
)

# Equal-frequency bins (quantile binning)
df['income_quartile'] = pd.qcut(
df['income'], q=4,
labels=['Q1_Low', 'Q2', 'Q3', 'Q4_High'],
duplicates='drop'
)

print(df['age_group'].value_counts())
print(df['income_quartile'].value_counts())
```

#80 KBinsDiscretizer (sklearn)

sklearn's KBinsDiscretizer is the pipeline-friendly version of binning. Supports uniform, quantile, and kmeans strategies.

```python
from sklearn.preprocessing import KBinsDiscretizer
import pandas as pd

kbd = KBinsDiscretizer(
n_bins=5,
encode='ordinal', # output as integers
strategy='quantile' # equal-frequency bins
)

df['income_bin'] = kbd.fit_transform(df[['income']]).astype(int)

# See the bin edges
print('Bin edges:', kbd.bin_edges_[0].round(2))
print(df[['income', 'income_bin']].head(10))
```

#81 Aggregation Features (Group Statistics)

Aggregate features capture the context of a record relative to its group — powerful for tabular data.

```python
```

```
# For each customer, compute stats about their city
city_stats = df.groupby('city')['income'].agg(
city_income_mean='mean',
city_income_std='std',
city_income_median='median',
city_count='count'
).reset_index()

# Merge back to original DataFrame
df = df.merge(city_stats, on='city', how='left')

# Relative income: how does this person compare to their city?
df['income_vs_city'] = df['income'] - df['city_income_mean']
df['income_zscore_city'] = (
(df['income'] - df['city_income_mean']) / (df['city_income_std'] + 1e-6)
)
print(df[['city','income','income_vs_city']].head())
```

#82 Cumulative / Rolling Window Features (Time-Series)

Rolling windows capture trends in time-series or sequential data — critical for forecasting models.

python

```
import pandas as pd

df = df.sort_values('date').reset_index(drop=True)

# Rolling mean (7-day)
df['sales_7d_avg'] = df['sales'].rolling(window=7, min_periods=1).mean()

# Rolling std (volatility)
df['sales_7d_std'] = df['sales'].rolling(window=7, min_periods=1).std()

# Cumulative sum
df['sales_cumsum'] = df['sales'].cumsum()

# Lag features (yesterday's value)
df['sales_lag1'] = df['sales'].shift(1)
df['sales_lag7'] = df['sales'].shift(7)

print(df[['date','sales','sales_7d_avg','sales_lag1']].head(10))
```

#83 Binary Flag Features

Simple 0/1 flags based on domain rules often outperform complex transformations — always try the obvious ones first.

python

```
# Business domain flags
df['is_high_value'] = (df['spend'] > df['spend'].quantile(0.75)).astype(int)
df['is_new_customer']= (df['tenure_months'] <= 3).astype(int)
df['has_multiple_products'] = (df['product_count'] > 1).astype(int)
df['is_weekend_buyer'] = df['day_of_week'].isin([5, 6]).astype(int)

# Missing value flags (captured before imputation)
for col in ['income', 'age', 'score']:
df[f'{col}_was_null'] = df[col].isnull().astype(int)

print(df.filter(like='is_').head())
```

#84 Text-derived Features (from a text column)

When you have a text column, extract simple numeric features before applying NLP — word count, char count, capitals ratio, etc.

```python
df['text'] = df['review'] # example text column

df['word_count'] = df['text'].str.split().str.len()
df['char_count'] = df['text'].str.len()
df['avg_word_len'] = df['char_count'] / (df['word_count'] + 1)
df['upper_ratio'] = df['text'].apply(
lambda x: sum(1 for c in str(x) if c.isupper()) / (len(str(x)) + 1)
)
df['has_exclaim'] = df['text'].str.contains('!').astype(int)
df['has_question'] = df['text'].str.contains('\\?').astype(int)
df['num_count'] = df['text'].str.count(r'\d')

print(df[['word_count','char_count','has_exclaim']].describe().round(2))
```

#85 Distance Features (from coordinates)

Haversine distance from a reference point (e.g., city center, store) is a powerful geospatial feature.

```python
import numpy as np

def haversine(lat1, lon1, lat2, lon2):
'''Compute distance in km between two GPS coordinates.'''
R = 6371 # Earth's radius in km
phi1, phi2 = np.radians(lat1), np.radians(lat2)
dphi = np.radians(lat2 - lat1)
dlambda = np.radians(lon2 - lon1)
a = np.sin(dphi/2)**2 + np.cos(phi1)*np.cos(phi2)*np.sin(dlambda/2)**2
return R * 2 * np.arctan2(np.sqrt(a), np.sqrt(1 - a))

# Distance from city center (e.g., Lahore: 31.5204, 74.3587)
ref_lat, ref_lon = 31.5204, 74.3587
df['dist_to_center'] = haversine(
df['lat'].values, df['lon'].values,
ref_lat, ref_lon
)
print(df['dist_to_center'].describe().round(2))
```

#86 Embedding Count Encoding (Rare Category Grouping)

Group rare categories (less than 1% frequency) into a single 'Other' bucket to reduce noise and cardinality.

```python
def group_rare_categories(series, threshold=0.01):
'''Replace categories with frequency < threshold with Other.'''
freq = series.value_counts(normalize=True)
rare = freq[freq < threshold].index
return series.replace(rare, '__Other__')

df['city_clean'] = group_rare_categories(df['city'], threshold=0.01)

print('Before:', df['city'].nunique(), 'unique values')
print('After: ', df['city_clean'].nunique(), 'unique values')
print(df['city_clean'].value_counts().tail(5))
```

#87 Feature Selection — Remove Low Variance Features

Features with near-zero variance carry almost no information. VarianceThreshold removes them automatically.

```python
```

```
from sklearn.feature_selection import VarianceThreshold
import pandas as pd

selector = VarianceThreshold(threshold=0.01) # remove if var < 1%

selector.fit(X_train)

# Which features were kept?
kept_cols = X_train.columns[selector.get_support()]
dropped = X_train.columns[~selector.get_support()]

print(f'Kept {len(kept_cols)}/{len(X_train.columns)} features')
print('Dropped (low variance):', dropped.tolist())

X_train_sel = selector.transform(X_train)
X_test_sel = selector.transform(X_test)
```

CHAPTER 6

Dimensionality Reduction

PCA, Incremental PCA, Kernel PCA, LDA, t-SNE and more — compressing features while retaining information

#88 PCA — Principal Component Analysis

PCA is the most common dimensionality reduction technique. It projects data onto orthogonal axes of maximum variance.

```python
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import pandas as pd

# CRITICAL: always scale before PCA
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_train)

# Keep components explaining 95% of variance
pca = PCA(n_components=0.95, random_state=42)
X_pca = pca.fit_transform(X_scaled)

print(f'Original features: {X_train.shape[1]}')
print(f'PCA components: {pca.n_components_}')
print(f'Variance explained: {pca.explained_variance_ratio_.sum():.3f}')
```

#89 PCA — Scree Plot (Choose Number of Components)

The scree plot shows how much variance each component explains. Look for the 'elbow' — the point where additional components add little.

```python
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import numpy as np

X_scaled = StandardScaler().fit_transform(X)
pca_full = PCA().fit(X_scaled)

cumvar = np.cumsum(pca_full.explained_variance_ratio_)

plt.figure(figsize=(10, 4))
plt.bar(range(1, len(cumvar)+1), pca_full.explained_variance_ratio_,
alpha=0.7, color='steelblue', label='Individual')
plt.plot(range(1, len(cumvar)+1), cumvar, 'r-o',
markersize=4, label='Cumulative')
plt.axhline(0.95, color='green', linestyle='--', label='95% threshold')
plt.xlabel('Components'); plt.ylabel('Variance Explained')
plt.title('PCA Scree Plot'); plt.legend()
plt.tight_layout(); plt.show()
```

#90 PCA — Visualize in 2D

Reduce to 2 components and visualize colored by target class — reveals if classes are linearly separable.

```python
```

```python
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

X_scaled = StandardScaler().fit_transform(X)
pca2 = PCA(n_components=2, random_state=42)
X_2d = pca2.fit_transform(X_scaled)

plt.figure(figsize=(8, 6))
for label in set(y):
mask = y == label
plt.scatter(X_2d[mask, 0], X_2d[mask, 1], label=f'Class {label}',
alpha=0.6, s=30)

plt.xlabel(f'PC1 ({pca2.explained_variance_ratio_[0]*100:.1f}%)')
plt.ylabel(f'PC2 ({pca2.explained_variance_ratio_[1]*100:.1f}%)')
plt.title('PCA — 2D Projection')
plt.legend(); plt.tight_layout(); plt.show()
```

#91 PCA — Loadings (Feature Importance in Components)

PCA loadings tell you which original features contribute most to each principal component — key for interpretability.

```python
import pandas as pd
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

X_scaled = StandardScaler().fit_transform(X)
pca = PCA(n_components=5, random_state=42)
pca.fit(X_scaled)

# Loadings DataFrame
loadings = pd.DataFrame(
pca.components_.T,
index=X.columns,
columns=[f'PC{i+1}' for i in range(pca.n_components_)]
)

print('Top features for PC1:')
print(loadings['PC1'].abs().sort_values(ascending=False).head(10))
```

#92 Incremental PCA (for Large Datasets)

IncrementalPCA processes data in mini-batches — use it when your dataset is too large to fit in memory.

```python
```

```
from sklearn.decomposition import IncrementalPCA
from sklearn.preprocessing import StandardScaler
import numpy as np

X_scaled = StandardScaler().fit_transform(X)

n_components = 20
batch_size = 500

ipca = IncrementalPCA(n_components=n_components)

# Fit in batches
for i in range(0, len(X_scaled), batch_size):
batch = X_scaled[i:i + batch_size]
if len(batch) >= n_components: # need at least n_components rows
ipca.partial_fit(batch)

X_reduced = ipca.transform(X_scaled)
print('Reduced shape:', X_reduced.shape)
```

#93 Kernel PCA (Non-linear Dimensionality Reduction)

Kernel PCA applies a kernel trick to find non-linear structure — use RBF kernel when data has non-linear patterns.

```python
from sklearn.decomposition import KernelPCA
from sklearn.preprocessing import StandardScaler

X_scaled = StandardScaler().fit_transform(X)

kpca = KernelPCA(
n_components=10,
kernel='rbf', # options: 'linear','poly','rbf','sigmoid','cosine'
gamma=0.1,
random_state=42,
fit_inverse_transform=True # allows reconstruction
)

X_kpca = kpca.fit_transform(X_scaled)
print('KernelPCA output shape:', X_kpca.shape)
```

#94 LDA — Linear Discriminant Analysis

LDA is a supervised dimensionality reduction method — it maximizes class separability. Use it for classification tasks where you know the labels.

```python
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler

X_scaled = StandardScaler().fit_transform(X_train)

# Maximum components = min(n_classes - 1, n_features)
n_components = min(len(set(y_train)) - 1, X_train.shape[1])

lda = LinearDiscriminantAnalysis(n_components=n_components)
X_lda = lda.fit_transform(X_scaled, y_train)

print('LDA output shape:', X_lda.shape)
print('Explained variance:', lda.explained_variance_ratio_.round(3))
```

#95 t-SNE — Non-linear 2D Visualization

t-SNE is purely a visualization tool (do NOT use for preprocessing). It reveals cluster structure invisible to PCA.

```python
from sklearn.manifold import TSNE
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

X_scaled = StandardScaler().fit_transform(X)

tsne = TSNE(
n_components=2,
perplexity=30, # try 5-50; larger datasets need higher values
learning_rate=200,
n_iter=1000,
random_state=42
)

X_tsne = tsne.fit_transform(X_scaled)

plt.figure(figsize=(8, 6))
for c in set(y):
mask = y == c
plt.scatter(X_tsne[mask,0], X_tsne[mask,1], label=f'Class {c}', alpha=0.6, s=20)
plt.title('t-SNE 2D Visualization')
plt.legend(); plt.tight_layout(); plt.show()
```

■ *t-SNE is stochastic and slow. Use on a sample (5000-10000 rows) first.*

#96 PCA in a Full ML Pipeline

PCA inside a Pipeline — the correct production pattern. Scaler, PCA, and model are fitted together without leakage.

```python
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import cross_val_score

pipe = Pipeline([
('scale', StandardScaler()),
('pca', PCA(n_components=0.95, random_state=42)),
('model', GradientBoostingClassifier(n_estimators=100))
])

scores = cross_val_score(pipe, X, y, cv=5, scoring='accuracy')
print(f'Accuracy: {scores.mean():.4f} +/- {scores.std():.4f}')
```

#97 Truncated SVD (Sparse Data / NLP)

TruncatedSVD works directly on sparse matrices (e.g., TF-IDF text features) — PCA doesn't. It's the basis of LSA for NLP.

```python
```

```
from sklearn.decomposition import TruncatedSVD
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd

# Text data -> TF-IDF sparse matrix
tfidf = TfidfVectorizer(max_features=5000)
X_tfidf = tfidf.fit_transform(df['review']) # sparse

# Reduce sparse matrix to 100 dense components (LSA)
svd = TruncatedSVD(n_components=100, random_state=42)
X_lsa = svd.fit_transform(X_tfidf)

print('Dense LSA shape:', X_lsa.shape)
print('Variance explained:', svd.explained_variance_ratio_.sum().round(3))
```

#98 Factor Analysis

Factor Analysis finds latent (hidden) variables that explain correlations. Use it when you believe there are unobserved constructs driving the data.

```python
from sklearn.decomposition import FactorAnalysis
from sklearn.preprocessing import StandardScaler
import pandas as pd

X_scaled = StandardScaler().fit_transform(X)

fa = FactorAnalysis(n_components=5, random_state=42)
X_fa = fa.fit_transform(X_scaled)

# Loadings: relationship between original features and latent factors
loadings_df = pd.DataFrame(
fa.components_.T,
index=X.columns,
columns=[f'Factor{i+1}' for i in range(5)]
)
print(loadings_df.round(3))
```

#99 Autoencoders for Dimensionality Reduction (Conceptual)

Autoencoders are neural network-based reducers. The encoder compresses features; the decoder reconstructs them. The bottleneck layer is your reduced representation.

```python
```

```
# Conceptual PyTorch autoencoder structure
# (install: pip install torch)
import torch
import torch.nn as nn

class Autoencoder(nn.Module):
def __init__(self, input_dim, latent_dim):
super().__init__()
self.encoder = nn.Sequential(
nn.Linear(input_dim, 64),
nn.ReLU(),
nn.Linear(64, latent_dim) # bottleneck
)
self.decoder = nn.Sequential(
nn.Linear(latent_dim, 64),
nn.ReLU(),
nn.Linear(64, input_dim),
nn.Sigmoid()
)

def forward(self, x):
z = self.encoder(x) # reduced representation
return self.decoder(z)

model = Autoencoder(input_dim=100, latent_dim=10)
print(model)
```

#100 Full ML Workflow — Everything Together

The complete end-to-end ML workflow combining EDA insight, feature engineering, imputation, outlier handling, and a PCA+model pipeline.

```
●●●  python
```

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
from sklearn.decomposition import PCA
from sklearn.ensemble import GradientBoostingClassifier

# 1. Load & split
df = pd.read_csv('data.csv')
X = df.drop('target', axis=1)
y = df['target']
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42, stratify=y
)

# 2. Define column groups
num_cols = X.select_dtypes(include='number').columns.tolist()
cat_cols = X.select_dtypes(include='object').columns.tolist()

# 3. Build preprocessor
num_pipe = Pipeline([
('impute', IterativeImputer(max_iter=10, random_state=42)),
('scale', StandardScaler())
])
cat_pipe = Pipeline([
('encode', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])
preprocessor = ColumnTransformer([
('num', num_pipe, num_cols),
('cat', cat_pipe, cat_cols)
])

# 4. Full pipeline: preprocess + PCA + model
full_pipe = Pipeline([
('prep', preprocessor),
('pca', PCA(n_components=0.95, random_state=42)),
('model', GradientBoostingClassifier(n_estimators=200, random_state=42))
])

# 5. Train and evaluate
full_pipe.fit(X_train, y_train)
cv_scores = cross_val_score(full_pipe, X_train, y_train, cv=5,
scoring='roc_auc')
test_acc = full_pipe.score(X_test, y_test)

print(f'CV ROC-AUC: {cv_scores.mean():.4f} +/- {cv_scores.std():.4f}')
print(f'Test Accuracy: {test_acc:.4f}')
```

Practice Consistently.

The gap between knowing and mastering is repetition.

---

100 snippets completed. Run each one. Break it. Fix it. Extend it.

That is how a data scientist is built.

Tech Stack Used: Pandas · NumPy · Matplotlib · Seaborn · Scikit-learn

Authored for self-paced ML skill development